

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – Sorting



NAMA: Cristian David Fernando

NIM: 109082500043

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Dasar Teori (Metode Pengurutan)

Laporan ini membedah dua jenis algoritma pengurutan data (Sorting), yaitu:

- **Selection Sort:** Algoritma ini bekerja dengan mencari nilai ekstrim (paling kecil atau paling besar) dari sekumpulan data terlebih dahulu, lalu menempatkannya (menukarnya) pada posisi yang seharusnya.
- **Insertion Sort:** Algoritma ini bekerja dengan mengambil satu nilai dan menyisipkannya pada posisi yang tepat pada kelompok data yang sudah terurut, biasanya dibantu dengan pergeseran data secara sequential search.

B. Guided

1. [InsertionSort]

```
package main

import "fmt"

func selectionSortArray(angka *[5]int) {
    var idx_min, i, j int
    for i = 0; i < len(angka)-1; i++ { //loop luar (iterasi
        sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari
            nilai ekstrim)
                if angka[j] <= angka[idx_min] { //sorting terkecil ke
                    terbesar
                        idx_min = j
                    }
            }
        if idx_min != i {
            angka[i], angka[idx_min] = angka[idx_min], angka[i]
        }
    }
}

func main() {
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
}
```

```

fmt.Println()

fmt.Println("=== SEBELUM DISORTING ===")
for i := 0; i < len(arrAngka); i++ {
    fmt.Print(arrAngka[i], " - ")
}

fmt.Println("\n=== SETELAH DISORITNG ===")
selecetionSortArray(&arrAngka)
for i := 0; i < len(arrAngka); i++ {
    fmt.Print(arrAngka[i], " - ")
}
}

```

Output :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14> go run "c:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Guided1\guided1.go"

Masukkan data angka ke-0 : 5
Masukkan data angka ke-1 : 7
Masukkan data angka ke-2 : 3
Masukkan data angka ke-3 : 9
Masukkan data angka ke-4 : 1

=== SEBELUM DISORTING ===
5 - 7 - 3 - 9 - 1 -
=== SETELAH DISORITNG ===
1 - 3 - 5 - 7 - 9 -
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14>

```

Penjelasan: Program ini meminta pengguna nya untuk memasukkan lima buah angka ke dalam sebuah *array*. Setelah data dimasukkan, program akan mengurutkan angka-angka tersebut dari nilai terkecil hingga terbesar (ascending) menggunakan algoritma *selection sort*. Terakhir, program menampilkan perbandingan isi data di dalam *array* sebelum dan sesudah proses pengurutan dilakukan.

2. [InsertionSortString]

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK       float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari
            //loop dalam mencari
            //nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil
                //ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println(" === SEBELUM SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
```

```

        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println(" === SETELAH SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

WEEK 12_MODUL 14

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Code + - [ ] [ ] ... [ ] [ ]

=== SEBELUM SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : Tian
NIM : 109082500043
IPK : 3.90
--- Data Mahasiswa Ke-1 ---
Nama : Bams
NIM : 109082500023
IPK : 3.80
--- Data Mahasiswa Ke-2 ---
Nama : Udin
NIM : 109082500001
IPK : 3.70
--- Data Mahasiswa Ke-3 ---
Nama : Budi
NIM : 109082500005
IPK : 3.60
--- Data Mahasiswa Ke-4 ---
Nama : Petot
NIM : 109082500010
IPK : 3.50
=====

=== SETELAH SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : Petot
NIM : 109082500010
IPK : 3.50
--- Data Mahasiswa Ke-1 ---
Nama : Budi
NIM : 109082500005
IPK : 3.60
--- Data Mahasiswa Ke-2 ---
Nama : Udin
NIM : 109082500001
IPK : 3.70
--- Data Mahasiswa Ke-3 ---
Nama : Bams
NIM : 109082500023

```

Penjelasan: Program berfungsi untuk menerima dan menyimpan input data lima mahasiswa, yang mencakup nama, NIM, dan nilai IPK. Setelah data dimasukkan, program menggunakan algoritma selection sort untuk mengurutkan daftar mahasiswa tersebut secara ascending (dari terkecil ke terbesar) berdasarkan nilai IPK-nya. Terakhir, program akan menampilkan kembali daftar mahasiswa tersebut untuk memperlihatkan perbandingan susunan data sebelum dan sesudah proses pengurutan dilakukan.

3. [SelSortArray]

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat
       F.S. array T terurut secara mengecil atau descending dengan
       INSERTION SORT */

    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

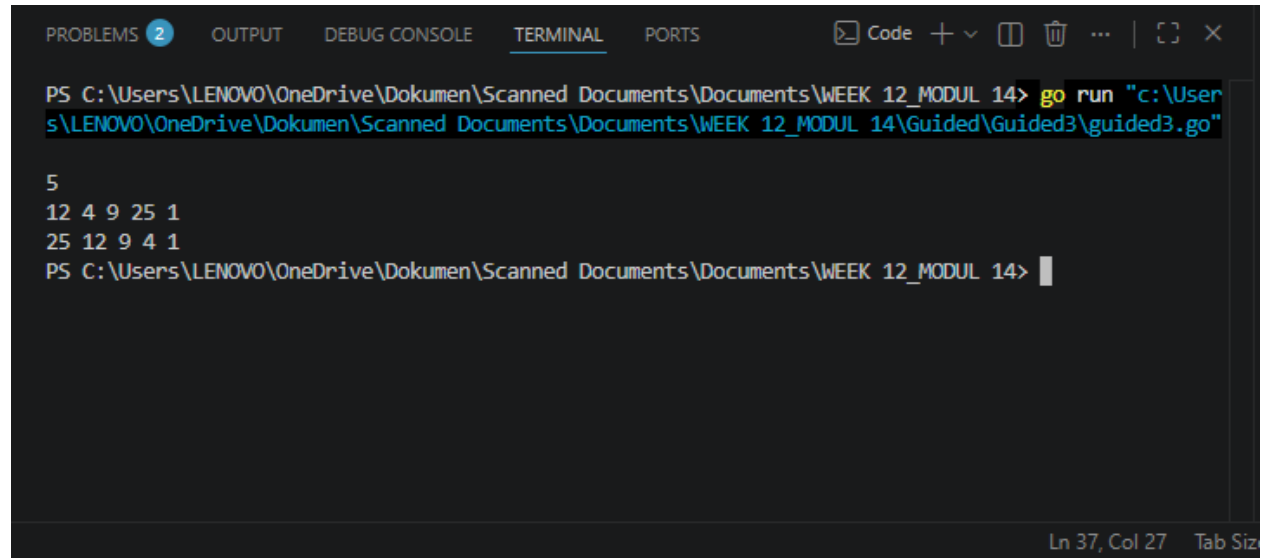
    insertionSort(&data, n)
```

```

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :



```

PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14> go run "c:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Guided3\guided3.go"

5
12 4 9 25 1
25 12 9 4 1
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14>

```

Penjelasan: Program ini adalah implementasi algoritma insertion sort dalam bahasa Go yang berfungsi untuk mengurutkan sekumpulan bilangan bulat dari nilai terbesar ke terkecil (descending). Program ini bekerja dengan cara menerima input berupa jumlah dan elemen data dari pengguna, memproses data tersebut menggunakan fungsi insertionSort, lalu mencetak hasil akhir yang sudah terurut ke layar.

4. [SelSortStruct]

```

package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

```



```

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

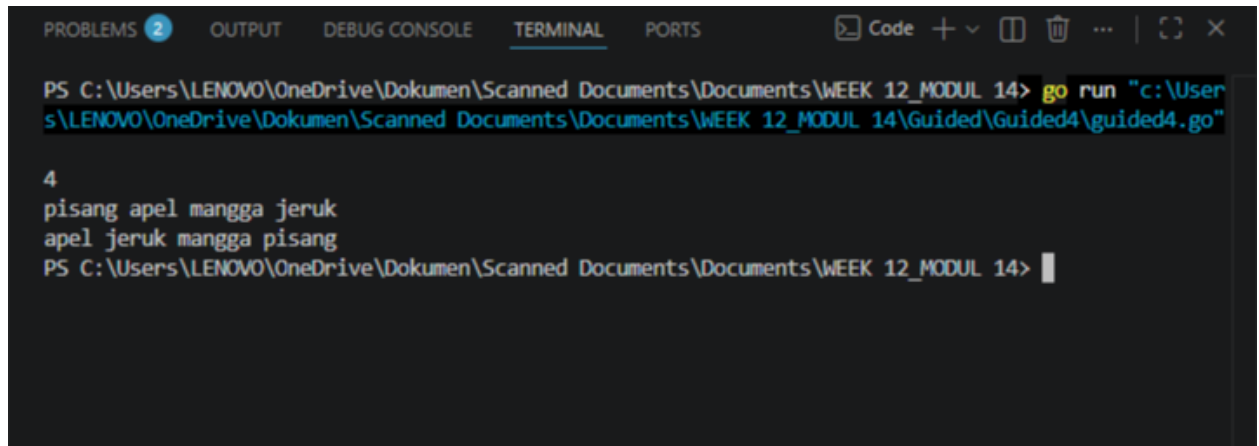
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :



```
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14> go run "c:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Guided4\guided4.go"

4
pisang apel mangga jeruk
apel jeruk mangga pisang
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14>
```

Penjelasan: Program ini merupakan implementasi algoritma Insertion Sort dalam bahasa Go yang berfungsi untuk mengurutkan sekumpulan data bertipe string secara alfabetis (ascending). Program akan membaca input berupa jumlah data (n) beserta teks-teksnya dari pengguna, lalu memprosesnya melalui fungsi `insertionSortString` dengan cara menyisipkan setiap elemen ke posisi yang tepat. Setelah proses pengurutan selesai, program akan mencetak kembali teks-teks tersebut ke layar dalam kondisi yang sudah terurut rapi.

C. Unguided

1. Soal Unguided 1

[Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya. **Masukan** terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array. **Keluaran** terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".]

Jawaban :

```
package main
```

```
import (
    "fmt"
)

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func main() {
    var arr []int
    var num int

    for {
        _, err := fmt.Scan(&num)
        if err != nil {
            break
        }

        if num < 0 {
            break
        }

        arr = append(arr, num)
    }

    if len(arr) == 0 {
        return
    }

    insertionSort(arr)

    for i, val := range arr {
        if i > 0 {
            fmt.Print(" ")
        }
    }
}
```

```

        fmt.Print(val)
    }
    fmt.Println()

    if len(arr) < 2 {
        fmt.Println("Data berjarak tidak tetap")
        return
    }

    jarak := arr[1] - arr[0]
    jarakTetap := true

    for i := 2; i < len(arr); i++ {
        if arr[i]-arr[i-1] != jarak {
            jarakTetap = false
            break
        }
    }

    if jarakTetap {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :

```

PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14> go run "c:\User
s\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Unguided\Unguided1\
unguided1.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14> go run "c:\User
s\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Unguided\Unguided1\
unguided1.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14>

```

Ln 73, Col 1 Tab Siz

Penjelasan :

[Program ini bisa membaca sekumpulan bilangan bulat dari input pengguna hingga menemukan angka negatif, lalu mengurutkannya secara menaik menggunakan algoritma insertion sort. Setelah menampilkan seluruh data yang sudah terurut, program kemudian memeriksa selisih antar elemen yang bersebelahan untuk mengevaluasi konsistensinya. Terakhir, program akan mencetak informasi apakah deret angka tersebut memiliki jarak yang konstan (membentuk barisan aritmetika) beserta nilai selisihnya, atau justru menyatakan bahwa jarak data tidak tetap.]

2. Soal Unguided 2

[Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini. **Masukan** terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari. **Keluaran** terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.]

Jawaban :

```
package main

import (
    "fmt"
)

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
```

```

    fmt.Scan(n)
    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
&pustaka[i].penulis, &pustaka[i].penerbit, &pustaka[i].eksemplar,
&pustaka[i].tahun, &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }
    maxIdx := 0
    for i := 1; i < n; i++ {
        if pustaka[i].rating > pustaka[maxIdx].rating {
            maxIdx = i
        }
    }
    b := pustaka[maxIdx]
    fmt.Println(b.judul, b.penulis, b.penerbit, b.tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }
    for i := 0; i < limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {

```

```

left := 0
right := n - 1
found := false
var b Buku

for left <= right {
    mid := (left + right) / 2
    if pustaka[mid].rating == r {
        b = pustaka[mid]
        found = true
        break
    } else if pustaka[mid].rating < r {
        right = mid - 1
    } else {
        left = mid + 1
    }
}

if found {
    fmt.Println(b.judul, b.penulis, b.penerbit, b.tahun,
b.eksemplar, b.rating)
} else {
    fmt.Println("Tidak ada buku dengan rating seperti itu")
}
}

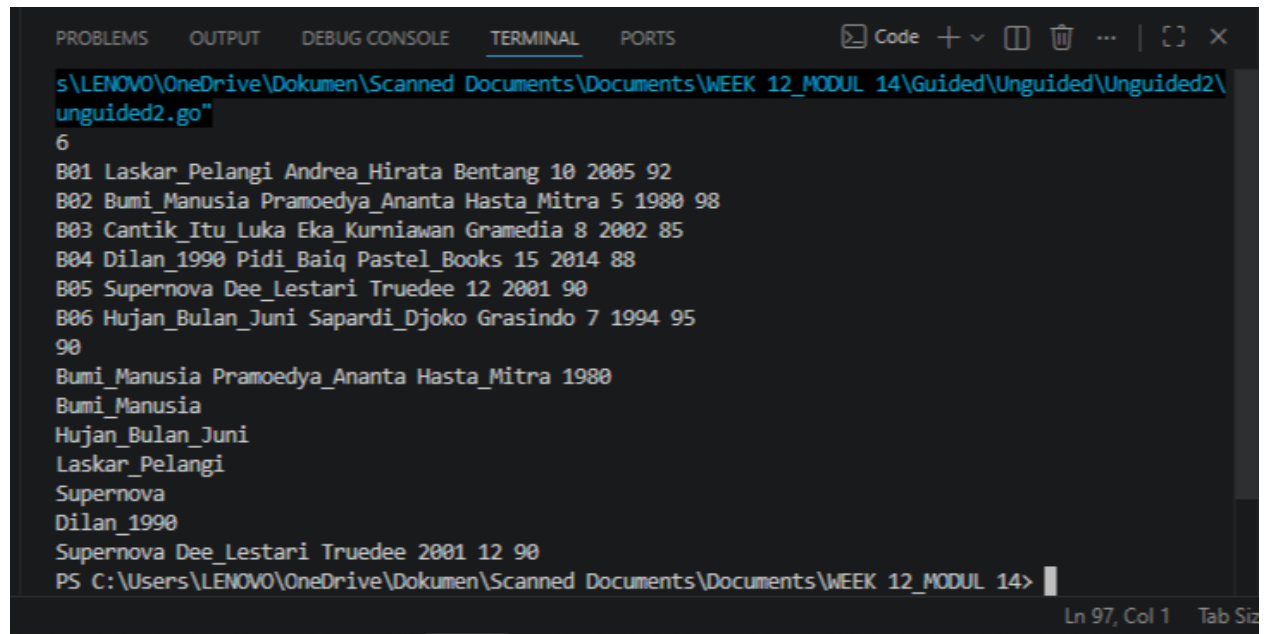
func main() {
    var pustaka DaftarBuku
    var n, ratingCari int

    DaftarkanBuku(&pustaka, &n)
    fmt.Scan(&ratingCari)

    CetakTerfavorit(pustaka, n)
    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)
    CariBuku(pustaka, n, ratingCari)
}

```

Output :



```
s:\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Unguided\Unguided2\unguided2.go"
6
B001 Laskar_Pelangi Andrea_Hirata Bentang 10 2005 92
B002 Bumi_Manusia Pramoedya_Ananta Hasta_Mitra 5 1980 98
B003 Cantik_Itu_Luka Eka_Kurniawan Gramedia 8 2002 85
B004 Dilan_1990 Pidi_Baiq Pastel_Books 15 2014 88
B005 Supernova Dee_Lestari Truedee 12 2001 90
B006 Hujan_Bulan_Juni Sapardi_Djoko Grasindo 7 1994 95
90
Bumi_Manusia Pramoedya_Ananta Hasta_Mitra 1980
Bumi_Manusia
Hujan_Bulan_Juni
Laskar_Pelangi
Supernova
Dilan_1990
Supernova Dee_Lestari Truedee 2001 12 90
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14>
```

Penjelasan :

[Program ini merupakan manajemen data perpustakaan sederhana yang membaca dan menyimpan informasi buku ke dalam struktur array. Program ini memproses data untuk menemukan buku ber-rating tertinggi, lalu mengurutkannya secara menurun berdasarkan rating menggunakan algoritma Insertion Sort guna menampilkan lima buku teratas. Terakhir, program mengimplementasikan algoritma Binary Search pada data yang telah terurut untuk mencari dan mencetak detail buku berdasarkan nilai rating spesifik yang diinputkan pengguna.]

3. Soal Unguided 3

[Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort. Masukan dimulai dengan sebuah integer $\cdot$ ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi $\cdot$ baris berikutnya selalu dimulai dengan sebuah integer $\diamond$ ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.]

Jawaban :

```
package main

import (
    "fmt"
)

func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

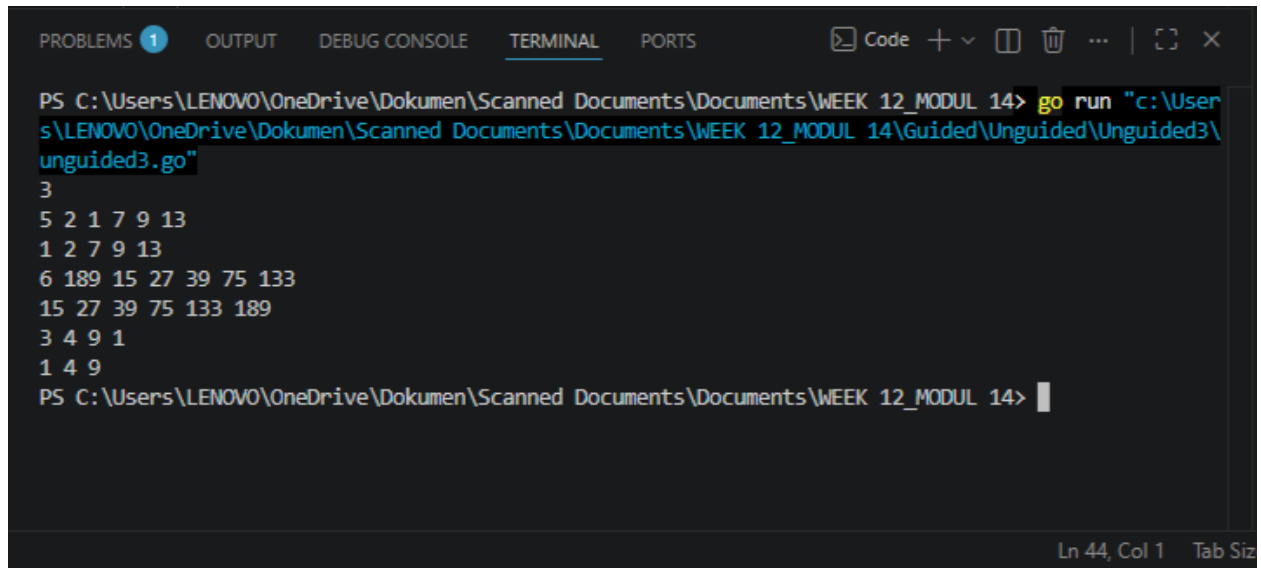
    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        arr := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }

        selectionSort(arr)

        for j := 0; j < m; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(arr[j])
        }
        fmt.Println()
    }
}
```

Output :



```
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14> go run "c:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Unguided\Unguided3\unguided3.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14>
```

Penjelasan :

[Program tersebut membaca input jumlah daerah beserta nomor-nomor rumah kerabat, kemudian menyimpannya ke dalam struktur data slice untuk masing-masing daerah. Setelah data terkumpul, program mengurutkan nomor rumah tersebut dari nilai terkecil ke terbesar menggunakan algoritma Selection Sort. Hasil akhir berupa deretan nomor rumah yang sudah terurut membesar akan dicetak ke layar untuk setiap daerah sesuai dengan permintaan soal.]

4. Soal Unguided 4

[Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil. Format Masukan masih persis sama seperti sebelumnya. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, dimasing-masing daerah.]

Jawaban :

```
package main

import (
    "fmt"
)

func sortAscending(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func sortDescending(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        maxIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[maxIdx] {
                maxIdx = j
            }
        }
        arr[i], arr[maxIdx] = arr[maxIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    var hasilAkhir []string

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var ganjil []int
```

```

var genap []int

for j := 0; j < m; j++ {
    var num int
    fmt.Scan(&num)
    if num%2 != 0 {
        ganjil = append(ganjil, num)
    } else {
        genap = append(genap, num)
    }
}

sortAscending(ganjil)
sortDescending(genap)

barisHasil := ""
for j, v := range ganjil {
    if j > 0 {
        barisHasil += " "
    }
    barisHasil += fmt.Sprintf("%d", v)
}

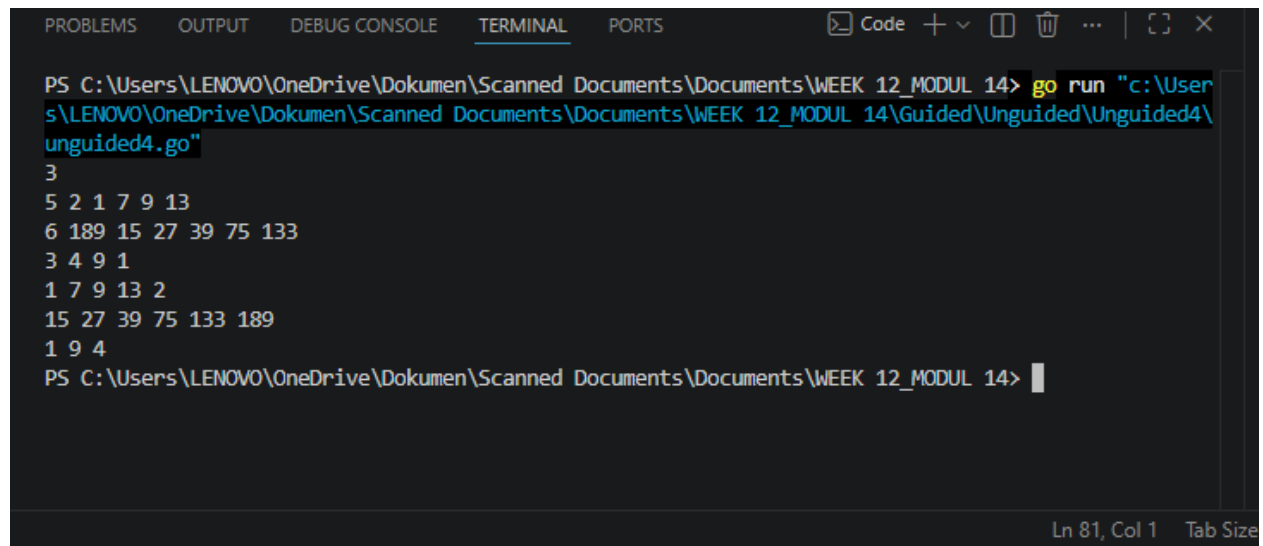
for _, v := range genap {
    if len(barisHasil) > 0 {
        barisHasil += " "
    }
    barisHasil += fmt.Sprintf("%d", v)
}

hasilAkhir = append(hasilAkhir, barisHasil)
}

for _, hasil := range hasilAkhir {
    fmt.Println(hasil)
}
}

```

Output :



```
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14> go run "c:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14\Guided\Unguided\Unguided4\unguided4.go"
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 1
1 7 9 13 2
15 27 39 75 133 189
1 9 4
PS C:\Users\LENOVO\OneDrive\Dokumen\Scanned Documents\Documents\WEEK 12_MODUL 14>
```

Penjelasan :

[Program ini memisahkan masukan nomor rumah ke dalam dua slice berbeda, yaitu untuk bilangan ganjil dan bilangan genap, tepat saat data dibaca. Setelah itu, data ganjil diurutkan secara membesar (ascending) dan data genap diurutkan secara mengecil (descending) menggunakan modifikasi algoritma pengurutan. Terakhir, program mencetak seluruh isi slice ganjil terlebih dahulu, lalu dilanjutkan langsung dengan isi slice genap dalam satu baris untuk setiap daerahnya.]

D. Kesimpulan

[Praktikum modul 14 ini membahas teori dan implementasi algoritma pengurutan data, yang secara khusus berfokus pada metode Selection Sort dan Insertion Sort menggunakan bahasa pemrograman go. Selama praktikum, metode pengurutan tersebut diterapkan untuk menyusun berbagai jenis tipe data, mulai dari tipe dasar seperti integer dan string hingga tipe data terstruktur (struct), baik secara membesar (ascending) maupun mengecil (descending). Pada akhirnya, penerapan algoritma ini juga dikombinasikan dengan logika pemecahan masalah untuk menyelesaikan studi kasus spesifik, seperti manajemen data rating buku perpustakaan dan pengurutan kustom nomor rumah ganjil-genap.]

